PracticAI

SIMPLE GUIDE

Page Index vs RAG

Context Retrieval Approaches

Retrieval

Context

1. What is Page Index Retrieval

Page Index means retrieving information based on **document structure** instead of semantic similarity.

Example Structure

Page 1 → Intro | Page 2 → Architecture | Page 3 → Chunking

Instead of searching a vector database, the system:

- **Searches the page index**
- **Finds the relevant page**
- **Retrieves the content of that page**
- **Breaks it slide by slide / section by section**

So the retrieval is **deterministic and structured.**

2. How Page Index Works

1. Document Parsing

Extract page numbers & headings, build index.

2. User Query

Query: "Explain embeddings" → Search keyword

3. Index Search

Match → Page 4 content retrieved.

4. Process Content

Break into Slides/Sections & send to LLM.

3. What is RAG?

RAG (Retrieval Augmented Generation) retrieves information using **semantic similarity search**.

Instead of using page numbers, it uses **vector embeddings**.

The Pipeline

Documents → Chunking → Embeddings → Vector DB

When a user asks a question:

- Query → Embedding
- Similarity Search
- Top K Chunks
- LLM Generation

Finds **meaningfully related content**, not exact matches.

4. How RAG Works

Step 1 — Chunking
Break documents into small pieces.

Step 2 — Embeddings
Convert chunks into vectors (e.g., [0.23, 0.89...]).

Step 3 — Vector Database
Store in Pinecone, Weaviate, FAISS, Chroma.

Step 4 — Query Retrieval
Embed user question & run similarity search.

Step 5 — LLM Generation
Retrieved chunks sent as context to the LLM.

5. Key Differences

Page Index

- → Keyword / page lookup
- → Document index storage
- → Simple architecture
- → Depends on doc structure
- → Very light infrastructure
- → Limited scalability

RAG

- → Semantic similarity
- → Vector database storage
- → Medium to complex
- → Better semantic matching
- → Requires embedding pipeline
- → Highly scalable

6. When to Use Page Index

Page index works best when the document is **well structured**.

Good Use Cases:

- **Slide Generators** (PDF → Index → Slides)
- **Educational Content** (Books, course materials)
- **Static Knowledge** (Manuals, structure matters)
- **Small Datasets** (< few hundred pages)

Benefits

Very fast • No embedding cost • Simple architecture • Easy debugging

7. When to Use RAG

RAG is better when **semantic understanding** is required.

Good Use Cases:

- **Enterprise Assistants** (Internal company docs)
- **Chatbots** (Customer support assistants)
- **Large Document Collections** (Millions of docs)
- **Unstructured Data** (Emails, logs, reports)

Benefits

Semantic search • Handles ambiguous questions • Works with large data • Flexible

8. Pros and Cons

Page Index

→ **Pros:**

→ No embedding cost

→ Fast retrieval

→ Easy to implement

→ **Cons:**

→ Depends on doc structure

→ Poor for large datasets

→ Limited search capability

RAG

→ **Pros:**

→ Semantic search

→ Handles vague questions

→ Works with large datasets

→ **Cons:**

→ Embedding cost

→ More infrastructure

→ Requires tuning

9. Simple Analogy

Page Index = Index Page of a Textbook

Embeddings → Page 42. You go directly to that exact page mechanically.

RAG = Smart Librarian

You ask: *"How do embeddings represent meaning?"*

The librarian finds relevant paragraphs across many different books and synthesizes them.

10. Quick Summary

- **Page Index:** Best for Small structured documents, Slide generation, Tutorials / courses
- **RAG:** Best for Chatbots, Large enterprise data, Semantic search

Ready to enhance retrieval?

Choose the approach that fits your data size & structure.